

Workflow tools compared

QCG-PilotJob, HyperQueue, Snakemake and FireWorks

Why compare tools

- Many small tasks inside one Slurm allocation need a tool that starts a task as soon as a core is free.
- QCG-PilotJob is one such tool. HyperQueue, Snakemake and FireWorks are three others.
- Each one expresses ”this task waits for those tasks” differently.
- The comparison: how the dependency is written, and how much of the allocation sits idle.

Hands-on

hands-on/alternatives/

The shared exercises

- **Example 4**: one average per CSV file, then an aggregation that must wait for every average.
- **Example 5**: sleeping tasks of different lengths, to measure idle time from the start and end time of each task.
- Same inputs and helper scripts. HyperQueue and Snakemake use 16 tasks;
  the file-backed FireWorks example uses one worker and one task.
- Job scripts: **average.sh** and **profile.sh** in a directory per tool.

HyperQueue

- A server and a worker run inside the allocation.
- One task per line of **files.txt**.

The dependency

- `--wait` returns when every task has finished; the aggregation is the next line.

```
# one task per line of files.txt
hq submit --each-line files.txt \
    --cpus=1 --wait \
    --stdout "$PWD/averages/%{TASK_ID}.csv" \
    -- bash -c \
    'python3 ../../example_04/average.py "$HQ_ENTRY"'


# then, after every task
cat averages/*.csv | sort > result.csv
```

SURF

Snakemake

- `snakemake --cores $SLURM_NTASKS` runs the rules on the cores of the allocation.

The dependency

- The input of **aggregate** lists every average file.

```
# Snakefile

rule aggregate:
    input:
        expand("averages/{name}.csv",
               name=NAMES),
    output:
        "result.csv",
    shell:
        "cat {input} | sort > {output}"
```

SURF

FireWorks

- The LaunchPad is a file under `$TMPDIR`; no MongoDB server.
- `rlaunch rapidfire`: one worker.
  A file LaunchPad allows no more.

The dependency

- `parents=averages` on the **aggregate** Firework.

```
# workflow.py

aggregate = Firework(
    ScriptTask.from_str(
        f"cat {HERE}/averages/*.csv"
        f" | sort > {HERE}/result.csv"),
    name="aggregate",
    parents=averages)
lp.add_wf(Workflow(averages + [aggregate],
                   name="average"))
```

SURF

Results: idle time

One run on 2-core SRC nodes with 2 slots, 32 tasks of example 5 at their original lengths. Not a general result.

Tool	Span	Idle
bash pipeline	266 s	36%
QCG-PilotJob	221 s	23%
HyperQueue	172 s	2%
Snakemake	172 s	2%

Results: averages of 100 files

The same 2-core SRC nodes, 2 slots.

Tool	Time
HyperQueue	23 s
Snakemake	46 s
QCG-PilotJob	3 min 12 s per version

What the idle time comes from

- The bash pipeline runs tasks in blocks of the slot count, so a block waits for its longest task.
- QCG-PilotJob: most of the 23% came from the module load each task did in that run. The examples leave **modules** out, so the tasks inherit the job's environment.
- HyperQueue and Snakemake start the next task when a slot frees up: 2% idle.
- Plot lanes show the CPU number a task reported at start. Tasks are not pinned, so overlapping bars can leave white space that is not idle time.
- The idle figure comes from the task start and end times.

SURF

How to choose

- **QCG-PilotJob** and **HyperQueue**: many tasks inside one allocation. QCG-PilotJob is driven from Python; HyperQueue from the command line, with a Python API as well.
- **Snakemake**: rules on files; a rerun builds only what changed.
- **FireWorks**: a database of workflows with states and reruns. Not for many small tasks: on Snellius one worker took 212 s for 1000 averages; Snakemake took 15 s on 16 cores.

SURF


SURF
Have fun! :)